

motorwatch

Surveillance open-set des régimes machine par signature de courant : du capteur au central, du central aux sites

Cahier de référence : conception, journal de réalisation, décisions et reconstruction

Portage industriel du cahier DriverV2 (partie V.2.1)

Projet SpikyPanda / CyanMycelium
11 juin 2026 : version 1.0

Document de travail confidentiel

Résumé pour lecteur non technique

De quoi s'agit-il ?

Ce dossier décrit la conception et la réalisation d'un système de surveillance de machines électriques capable de répondre, sans aucune configuration préalable, à la question : « **cette machine est-elle en train de changer de comportement, et pourquoi ?** »

Le principe : une pince de courant (un capteur bon marché, posé sur un câble d'alimentation, sans rien démonter) observe en continu le courant que consomme un moteur. La forme de ce courant est une signature : un moteur à vide, un moteur chargé, un moteur dont un composant commence à se dégrader ne « consomment » pas de la même façon. Un petit programme d'intelligence artificielle transforme quelques dizaines de millisecondes de courant en une empreinte mathématique, et regroupe les empreintes qui se ressemblent. La machine découvre ainsi TOUTE SEULE ses « régimes de fonctionnement », sans qu'on lui ait jamais dit combien il en existe ni à quoi ils ressemblent : c'est ce qu'on appelle le fonctionnement en « ensemble ouvert » (open-set).

Quand un régime jamais vu apparaît, le capteur lève une alarme vers un poste central. Le central peut alors lui téléverser, à chaud et sans redémarrage, de petits modèles d'IA de diagnostic : « est-ce une surcharge ? un déséquilibre d'alimentation ? une barre rotorique cassée ? ». Le verdict permet de poser une étiquette (« barre cassée ») qui est mémorisée localement par le capteur ET partagée entre les sites industriels : ce qu'un site a appris, les autres le savent. C'est le portage industriel direct de la recette validée dans le projet DriverV2 (regroupement anonyme de conducteurs) : son cahier de référence identifiait ce portage comme l'application transverse au meilleur rapport information/coût (partie V.2.1).

Ce qui a été réalisé

Le travail couvre la chaîne complète, du moteur simulé jusqu'au partage d'étiquettes entre sites, en deux sous-projets et sept volets :

- **Sous-projet 1 : moteur à courant continu R385** : un petit moteur de modélisme servant de banc d'essai réaliste (mêmes paramètres que le banc physique : résistance, inductance, inertie), alimenté par un onduleur PWM simulé et lu à travers un capteur de courant bruité. Le scénario complet y est démontré : trois régimes de charge successifs, deux alarmes « nouveau régime », diagnostics poussés à chaud, étiquettes fédérées entre deux sites.
- **Sous-projet 2 : moteurs triphasés** : les moteurs brushless existants (PMSM/BLDC) sont couverts, et surtout un **moteur asynchrone à cage d'écureuil a été modélisé de zéro** (il n'existait pas dans la plateforme), avec le défaut industriel emblématique : les barres rotoriques cassées. La cassure est simulée physiquement (asymétrie de résistance du rotor) et produit exactement la signature attendue par la littérature : des raies latérales dans le spectre du courant, dont l'amplitude croît avec le nombre de barres cassées. Le scénario y est rejoué : barres cassées EN COURS DE FONCTIONNEMENT, une seule alarme, diagnostic différentiel correct (« barre cassée », et non « changement de charge »), étiquette propagée à un second site.
- **Des briques génériques rangées au bon endroit** : selon la règle fixée en début de projet, tous les composants réutilisables sont des « nœuds » de l'éditeur graphique nodeeditor v2 (et non du code privé de l'application) : générateur de couple de charge, détecteur de régime établi, multiplexeur de phases, fenêtres multi-canaux, et un tout nouveau plugin d'apprentissage automatique (clustering open-set). L'application elle-même ne contient QUE la logique métier.
- **Une réparation profonde du moteur d'exécution** : faire circuler des tenseurs à travers un modèle ONNX inséré comme nœud d'un graphe (le « streaming fractal ») ne fonctionnait pas : le mécanisme

prévu était cassé depuis sa création et n'avait jamais été utilisé. Sa remise en état a au passage réparé 7 tests de la plateforme qui échouaient AVANT ce projet.

- **Le téléversement de modèles durci** : empreinte cryptographique (SHA-256), vérification du contrat d'entrées/sorties, bascule atomique « double banque » : un modèle rejeté laisse l'ancien parfaitement intact, y compris pour les erreurs subtiles (mauvais nombre de scores pour la liste de causes).
- **Quatre graphes de démonstration versionnés** : éditables dans l'éditeur graphique, rechargeables, et conçus pour la réutilisation : le bloc moteur est indépendant de sa charge (un seul fil à remplacer pour changer la loi de charge), et un test automatique recharge ces fichiers « comme l'éditeur » et vérifie qu'ils tournent.
- **Une revue contradictoire systématique** : une trentaine d'agents de relecture ont attaqué le code livré ; chaque défaut allégué a été contre-vérifié par reproduction exécutée. Neuf défauts réels que les tests rataient ont été confirmés puis corrigés (avec, pour chacun, un test qui échoue avant le correctif et passe après).

Le travail en chiffres

Indicateur	Valeur
Fichiers créés / modifiés	47 créés (8 877 lignes TypeScript) / 22 modifiés (+1 605 / -859 lignes)
Nouveaux nœuds d'éditeur	6 (couple de charge, moteur à induction, détecteur de régime établi, multiplexeur, transposition, clusterer open-set) + extension multicanal du tampon de fenêtres
Nouveau plugin	@spikypanda/plugin-ml (clustering open-set, bibliothèque + nœud)
Tests automatiques du périmètre	15 suites, 139 tests, tous verts
Tests du dépôt complet	1 193 verts / 1 198 (les 5 échecs restants sont antérieurs au projet, preuve à l'appui)
Tests préexistants RÉPARÉS par le projet	7 (perte de jetons sur les ports frontière du moteur d'exécution)
Défauts confirmés par la revue contradictoire puis corrigés	9 (3 majeurs, 6 moyens) + 8 mineurs
Graphes de démonstration versionnés	4 fichiers .spikypanda (format v3 de l'éditeur)
Modèles ONNX livrés en binaire	0 : tous les modèles de test sont synthétisés à la volée, à poids fixés

Les résultats, en langage courant

- Sur le banc R385 : trois régimes de charge appliqués successivement sont découverts comme **trois profils distincts** ; le premier (mise en service) est silencieux par contrat, les deux suivants lèvent **exactement une alarme chacun**, sont diagnostiqués (« palier de surcharge », « loi de ventilateur ») et leurs étiquettes survivent à l'export, à l'import et à la fusion entre deux sites.
- Sur le moteur à cage : deux barres cassées sur 28 produisent une raie latérale basse à **3,5 % du fondamental** (7 % pour quatre barres : la proportionnalité au nombre de barres attendue par la théorie est mesurée). La cassure déclenchée EN MARCHÉ est détectée comme un régime nouveau, et le diagnostic différentiel répond « barre rotorique cassée » là où un simple changement de charge, testé en contre-épreuve, est correctement étiqueté « changement de charge ».

- Dans l'éditeur graphique : le graphe de démonstration chargé tel quel devient un **moniteur vivant** dès qu'on y téléverse l'encodeur : il découvre un régime quand on change la charge, en direct. Vérifié au navigateur : 156 types de nœuds, zéro erreur console.

Honnêteté sur le périmètre : l'encodeur (le réseau qui transforme le courant en empreinte) utilisé dans les tests et la démo est **synthétique à poids fixés**, mais à la TOPOLOGIE EXACTE de la référence DriverV2 : trois étages de convolution 1D, moyennage global, tête dense (493 paramètres, contre 5 224 pour l'encodeur DriverV2 entraîné). Il démontre l'architecture ET valide le jeu d'opérateurs cible (Conv, GAP) sur toute la chaîne (export, téléversement validé, streaming dans l'éditeur), pas la performance d'un modèle entraîné. L'entraînement d'un encodeur industriel réel et sa campagne « simulation vers réel » sont exactement le travail que le cahier DriverV2 décrit en partie V.3 : c'est l'étape suivante naturelle, pas un oubli.

Les questions encore ouvertes

- **L'encodeur entraîné** : reproduire pour le courant moteur ce qui a été fait pour la conduite dans DriverV2 (simulateur statistiquement réaliste, entraînement contrastif, transfert gelé mesuré sur données réelles, par exemple le jeu public UFU de barres cassées).
- **Le transport réel** : le protocole capteur-central est complet mais câblé en mémoire (in-process) ; le branchement MQTT/BLE et le découpage en trames du téléversement restent à faire.
- **Le re-clustering périodique** : le déclencheur existe sur le nœud (plan de contrôle) mais n'est pas encore exposé comme outil du protocole ; sur un capteur allumé des mois, c'est le mécanisme de résorption de la sur-segmentation.
- **Le portage microcontrôleur** : comme pour DriverV2, tout est dimensionné pour l'embarqué (modèles de quelques kilo-octets, clustering en arithmétique pure) mais la compilation sur cible reste à faire.

Comment lire ce dossier

La **partie I** décrit le système tel qu'il est construit : la chaîne du capteur, les moteurs simulés, le moteur d'exécution, le central et la fédération. La **partie II** est le journal de réalisation : chaque décision, chaque découverte, chaque échec instructif, dans l'ordre où ils sont arrivés, y compris les neuf défauts trouvés par la revue contradictoire et la façon dont ils ont été confirmés puis corrigés. La **partie III** donne tout ce qu'il faut pour reconstruire et étendre le système (fichiers, commandes, contrats d'interface, calibrations). Les **annexes** contiennent la correspondance brique à brique avec le cahier DriverV2, le détail des défauts corrigés, les limites connues et un glossaire.

Table des matières

Résumé pour lecteur non technique	2
De quoi s'agit-il ?	2
Ce qui a été réalisé.....	2
Le travail en chiffres.....	3
Les résultats, en langage courant	3
Les questions encore ouvertes.....	4
Table des matières.....	5
PARTIE I : Le système	7
I.1 Le scénario et l'héritage DriverV2	7
I.2 La règle de placement : plugins, core, application	7
I.3 Le contrat d'acquisition et la chaîne du capteur (edge)	7
I.4 Les moteurs simulés.....	8
I.5 Le streaming fractal : la réparation de core.....	9
I.6 Le téléversement de modèles : la double banque	10
I.7 Le central : diagnostic différentiel et labellisation.....	10
I.8 La fédération entre sites	10
I.9 Le protocole capteur-central.....	10
I.10 Les graphes de démonstration.....	11
PARTIE II : Journal de réalisation	12
II.1 Cadrage : trois directives qui ont fait l'architecture	12
II.2 Première vague : cinq chantiers de fondation en parallèle	12
II.3 L'application : le métier, et un choix d'architecture défendu	13
II.4 La découverte qui a réorienté la semaine : le streaming fractal était cassé.....	14
II.5 Les graphes : deux leçons de câblage et un test phare	14
II.6 Sous-projet 2 : la cage d'écureuil de bout en bout.....	15
II.7 Vérification globale et preuve de non-régression.....	15
II.8 La revue contradictoire : neuf défauts réels que les tests rataient	15
II.9 État final	16
II.10 Retour d'usage immédiat : sept défauts signalés aux premiers essais	17
II.11 La contrainte d'acquisition : le capteur reprend son horloge	18
II.12 La grenouille bouillie : l'ancre de référence.....	19
PARTIE III : Guide de reconstruction	21
III.1 Où vivent les fichiers	21
III.2 Construire et vérifier	21
III.3 Les contrats d'interface.....	21
III.4 Synthétiser les modèles ONNX de test et de démonstration.....	22
III.5 Les calibrations qui comptent.....	23
III.6 Étendre le système	23
ANNEXES	24
Annexe A : correspondance brique à brique avec le cahier DriverV2	24
Annexe B : les neuf défauts confirmés par la revue, et leurs correctifs.....	24

Annexe C : allégations réfutées (et pourquoi c'est important)	25
Annexe D : limites connues et travaux futurs	25
Annexe E : glossaire.....	26
Annexe F : inventaire des suites de tests du périmètre	26

PARTIE I : Le système

I.1 Le scénario et l'héritage DriverV2

motorwatch est le portage industriel de la recette DriverV2 : **regroupement comportemental en ensemble ouvert, sur séries temporelles, embarqué, sans étiquettes**. On remplace « conducteur » par « régime de fonctionnement ou famille de défaut », et « centrale inertielle du véhicule » par « courant statorique du moteur ». Les quatre briques génériques identifiées par le cahier DriverV2 (partie V.1) se retrouvent une à une : l'acquisition événementielle devient une porte de régime ÉTABLI (on capture les états stationnaires, pas les transitoires : la sémantique est inversée mais la machine à états est la même) ; l'encodeur boîte noire ONNX est conservé tel quel dans son contrat ; le clustering anonyme à k émergent est porté ligne à ligne ; la méthodologie (simulateur paramétré, transfert gelé, résultat négatif documenté) structure le journal de la partie II.

Le scénario opérationnel en une phrase : un capteur de courant découvre seul un régime nouveau et alerte le central ; le central téléverse au capteur, à chaud, des modèles ONNX de diagnostic ; le verdict différentiel permet de labelliser le régime ; le label est référencé localement par le capteur et fédéré entre les sites.

I.2 La règle de placement : plugins, core, application

Décision structurante prise en début de projet (et appliquée sans exception) : **tout nœud générique va dans un plugin** (compatible nodeeditor v2, donc utilisable par n'importe quelle application graphique), **tout ajout 100 % générique de niveau framework va dans core**, et l'application ne garde QUE la logique métier. Conséquence : motorwatch n'introduit aucun nœud privé, et la moitié de sa valeur est immédiatement réutilisable ailleurs.

Pièce	Où elle vit	Identifiant
Générateur de couple de charge (5 profils)	plugin physics	Physics.Mechanical.Load:torque
Moteur asynchrone à cage d'écureuil (barres cassées)	plugin physics	Physics.Electric.Motor.Induction:dynamic
Détecteur de régime établi (porte à hystérésis)	plugin dsp	DSP.Detect:steadystate
Multiplexeur de phases (N flottants vers tenseur [N])	plugin dsp	DSP.Stream:mux
Fenêtres multi-canaux [T,C] (extension du tampon)	plugin dsp	DSP.Stream:buffer
Transposition de tenseur (vers (1,C,T) pour l'ONNX)	plugin dsp	DSP.Tensor:transpose
Clustering open-set (bibliothèque + nœud)	plugin ml (NOUVEAU)	ML.Cluster:online
Téléversement de modèle validé (double banque)	plugin onnx	OnnxModelGraph.loadModelValidated
Streaming fractal par nom de port	core (RuntimeGraph)	ports frontière
Catalogue, fédération, station, protocole, façade capteur	application motorwatch	logique métier seulement

I.3 Le contrat d'acquisition et la chaîne du capteur (edge)

Le capteur possède sa propre horloge d'échantillonnage : c'est une contrainte de conception posée par le donneur d'ordre (profil d'acquisition IEC 61430 / 61407 tel que transmis) et c'est la pratique standard de la surveillance de machines : échantillonnage à **10,24 kHz** (le facteur 2,56 x Fmax classique, Fmax = 4 kHz d'analyse), livraison par **blocs de 2 048 échantillons, soit 200 ms**, résolution spectrale fs/N = 5 Hz (ce qui résout proprement les bandes latérales de barres cassées à +/- 10 Hz). Le nœud générique DSP.Acquire:daq matérialise ce contrat : son horloge est fondée sur le TEMPS SIMULÉ (les instants k/fs), donc totalement découplée de la cadence des ticks de session ; il publie chaque bloc (tenseur [2048]) ET le RMS du bloc. Le RMS par bloc est le scalaire naturel de la porte de régime : à la cadence bloc (5 Hz), l'ondulation PWM et le bruit capteur sont moyennés, plus besoin de lissage au niveau échantillon. Un filtre anti-repliement 1 pôle (coupure Fmax) tient lieu de filtre ADC.

Le pipeline embarqué, construit uniquement avec des nœuds de plugins, devient :

```
courant i(t) (capteur analogique bruite)
-> DSP.Acquire:daq          horloge PROPRE 10,24 kHz, blocs [2048] / 200 ms + RMS
  |-> rms -> DSP.Detect:steadystate  porte de regime a la cadence bloc (5 Hz)
  |-> block -> DSP.Frame(64, hop 2048) -> fenetre [1,64] (cliche de tete de bloc)
      -> transpositions -> (1,1,64) -> encodeur ONNX -> ML.Cluster:online
  |-> block -> fenetre de Hann -> FFT 2048 points (bins de 5 Hz) -> spectre MCSA
```

Côté façade capteur (variante MCU), la même discipline s'applique avec en plus la quarantaine d'hystérésis et la suppression du démarrage à froid ; dans le graphe d'éditeur, le tout premier bloc (transitoire de démarrage) crée légitimement un profil « mise en route » visible.

La porte de régime établi

Machine à états à hystérésis sur un scalaire (le courant, ou son enveloppe en alternatif) : un échantillon est « stable » si son écart à une moyenne mobile exponentielle reste sous un seuil relatif epsilon ; l'état STABLE s'ouvre après settle échantillons stables consécutifs et se referme après breakHold échantillons instables. Pendant la confirmation de rupture, la porte laisse passer jusqu'à breakHold-1 échantillons post-transition : c'est son contrat (testé). C'est la façade capteur qui restaure l'invariant « aucune fenêtre ne chevauche un changement de régime » avec une petite file de quarantaine de breakHold-1 échantillons, vidée avant la remise à zéro du tampon sur chaque transition (défaut trouvé par la revue, partie II.8).

Le clustering open-set

Portage ligne à ligne de la brique 4 de DriverV2. Assignment en ligne par distance cosinus aux centroïdes :

$$d(e, c) = 1 - (e / ||e||) \cdot (c / ||c||)$$

$d < \text{assign_thr}$: assignment au profil le plus proche (et mise à jour EMA du centroïde si $d < \text{update_thr}$) ; sinon : création d'un profil, c'est le signal « nouveau régime ». Seuils calibrés hérités de DriverV2 : assign 0,05, update 0,02, alpha 0,15. Le re-clustering par lots (agglomératif à seuil ABSOLU link_thr = 0,06, k_max = 4) reste disponible pour résorber la sur-segmentation ; son déclencheur est un port de **plan de contrôle** (_recluster) : la revue a montré qu'en port de données il bloquait puis plantait la session (partie II.8). L'historique est borné (512 entrées en anneau) : un capteur allumé des mois ne grossit pas. La sortie alarm émet, uniquement à la création d'un profil, une trame compatible avec le bus d'alertes de l'éditeur : sévérité « warn » (la grammaire du bus normalise toute autre valeur, y compris « warning », vers « info » : pièges d'intégration documentés en II.2).

1.4 Les moteurs simulés

Sous-projet 1 : le R385 (courant continu)

Le nœud `Physics.Electric.Motor.DC:dynamic` existait : c'est un nœud INTÉGRABLE (états $[i, \omega]$ intégrés par le solveur RK4 adaptatif de la session, entrées V et τ_{load} en ports « signal » à maintien d'ordre zéro). Les valeurs par défaut ne sont PAS celles d'un R385 : tous les bancs (tests et graphes) posent explicitement $R = 1,22 \text{ ohm}$, $L = 1 \text{ mH}$, $K_e = K_t = 8,22 \text{ mN.m/A}$, $J = 6e-7 \text{ kg.m}^2$, $b = 1,03e-6$. La chaîne de démonstration reprend le graphe de référence fourni par l'utilisateur : potentiomètre, onduleur PWM (12 V, 10 kHz, bipolaire), moteur, capteur de courant LEM (bruit, résolution, bande passante).

Sous-projet 2 : la machine asynchrone à cage d'écureuil (créée pour ce projet)

Modèle en vecteurs d'espace dans le repère stationnaire alpha-beta, flux comme variables d'état :

```
i_s = (Lr.psi_s - Lm.psi_r)/D      i_r = (Ls.psi_r - Lm.psi_s)/D      D = Ls.Lr - Lm^2
dpsi_s/dt = v_s - Rs.i_s
dpsi_r/dt = -R_r(theta).i_r + j.P.omega.psi_r
Te = (3/2).P.(Lm/Lr).(psi_r_a.i_s_b - psi_r_b.i_s_a)
J.domega/dt = Te - b.omega - (tau_load + tau_fautes)
```

La **cassure de barres** est une asymétrie de résistance rotorique FIXE DANS LE ROTOR : la matrice $R_r(\theta)$ vaut $R_r(I + \delta \cdot \text{Proj}(a))$ où $\text{Proj}(a)$ projette sur l'axe d'asymétrie $a = P \cdot \theta$ (angle électrique du rotor) et $\delta = \text{bar_severity} \times \text{broken_bars} / \text{total_bars}$. C'est le modèle localisé standard : il produit dans le courant statorique les raies latérales à $f \cdot (1 \pm 2s)$ dont l'amplitude suit k/N , exactement la signature MCSA décrite par la littérature (et par les notes de recherche du dépôt, `motor-current-mcsa-principles.md` : environ 2,9 % de modulation pour 1 barre sur 34). Mesuré sur le modèle au taux déclaré (~9,8 kHz) : raie basse à 3,5 % du fondamental pour 2 barres sur 28, 6,7 % pour 4 barres (rapport 1,9x). La sortie `slip` publie le glissement (borné $[-1, 2]$: la plage couvre la génératrice ET le freinage en contre-courant).

Intégration : Euler explicite dans `fire()` (cohérent avec la famille brushless existante), avec un taux requis auto-déclaré calé sur le VRAI pôle rapide du modèle, $\tau_e = D/(R_s.L_r + R_r.L_s)$, marge $40/\tau_e$, plancher $80 \times f_{\text{alimentation}}$: la première heuristique (qui ignorait R_r) sous-déclarait le taux d'un facteur 8 et faussait le glissement de moitié (défaut trouvé et corrigé par la revue, partie II.8).

Le couple de charge : la pièce qui manquait

Aucun nœud ne produisait de couple de charge variable : le graphe de référence le composait à la main (canal de retour Z-1 + deux multiplications pour $K \cdot \omega^2$). `Physics.Mechanical.Load:torque` fournit cinq profils : constant, échelon (T0 vers T1 à t_{step} : l'apparition d'un régime), rampe (dérive), quadratique (loi de ventilateur/pompe), périodique (modulation mécanique). La loi quadratique est SIGNÉE : $\tau = k \cdot \omega \cdot |\omega|$, et la sortie est bornée « ne jamais entraîner » ($\tau \cdot \omega \geq 0$ pour les profils dépendant de la vitesse) : la version naïve $k \cdot \omega^2$ violait la passivité en rotation inverse et divergeait en temps fini (revue, partie II.8).

1.5 Le streaming fractal : la réparation de core

Le scénario exige qu'un modèle ONNX, inséré comme Nœud d'un graphe (le nœud « `spk.onnx:model` » de l'éditeur), consomme et produise des tenseurs en flux. Constat au moment de câbler : le mécanisme prévu (canaux frontière à extrémité pendante, `withInputPort/withOutputPort`) était cassé depuis sa création et n'était utilisé NULLE PART ; le routage d'entrée comparait le mauvais nom de port ; et le chemin classique `infer()` reposait sur des invariants que des canaux frontière naïfs auraient brisés.

La remise en état, entièrement dans `core` et le plugin `onnx`, établit le contrat de câblage suivant :

- VERS un nœud modèle : le **toSlot** du canal est le NOM du tenseur d'entrée ONNX (ex. « `current_window` »). Le modèle tire « tout ou rien » : il ne s'exécute que si chaque entrée déclarée a

reçu un jeton à ce tick (un modèle partiellement câblé jette les jetons et saute l'exécution, au lieu d'empoisonner la session : défaut majeur corrigé).

- DEPUIS un nœud modèle : le **fromSlot** du canal est le NOM du tenseur de sortie (ex. « embedding »). La valeur est lue une fois puis publiée vers TOUS les canaux parents qui la demandent (le fan-out vers plusieurs consommateurs n'alimentait que le premier : défaut majeur corrigé).
- Les accesseurs `inputPorts/outputPorts` du nœud modèle exposent les noms de tenseurs après chargement : l'éditeur les dessine comme des ports câblables.
- Le chemin classique `infer()` reste intact octet pour octet ; l'injection `pendingInput PERSISTE` sur les nœuds source purs (les boucles d'entraînement en dépendent : leçon apprise à nos dépens, partie II.8) et n'est consommée à la lecture que sur la branche frontière.

Bénéfice collatéral : cette remise en état a réparé **7 tests préexistants du dépôt** (`nested.test.ts`) qui échouaient AVANT le projet : la perte de jetons sur les canaux de sortie pendants en était la cause racine.

I.6 Le téléversement de modèles : la double banque

`loadModelValidated(bytes, options)` : vérification SHA-256 AVANT le parsing (insensible à la casse), construction en zone de transit, validation du contrat (forme d'entrée déclarée avec jokers pour les dimensions dynamiques, nombre de sorties, et FORME de sortie : c'est elle qui garantit « un score par cause » avant de basculer), puis bascule atomique. Tout échec laisse le modèle précédent ET ses métadonnées intacts ; la raison est exposée. Côté capteur, le banc de diagnostic est tenu par un petit adaptateur (kernel privé de l'application) : la session d'exécution fige la topologie à la construction, donc remplacer les nœuds INTERNES d'un nœud fractal en cours de session serait invisible ; l'adaptateur garde la frontière stable et permute les banques derrière elle. Le SHA du dernier téléversement réussi est mémorisé côté central : une alarme répétée ne re-téléverse pas les mêmes octets (outil `diagnostic_run` sans transfert).

I.7 Le central : diagnostic différentiel et labellisation

La station s'abonne aux alarmes. Sur `NEW_REGIME` : recherche d'abord dans son catalogue (un régime déjà connu est re-labellisé sans diagnostic : le chemin rapide) ; sinon, téléversement du modèle de diagnostic enregistré, exécution sur la dernière fenêtre capturée, et verdict différentiel :

```
{ topCause, score, runnerUp, margin = score(top) - score(runnerUp) }
```

Si la marge dépasse le seuil (0,25 par défaut), le label est la cause gagnante ; sinon « `regime_k_unlabeled` » (l'aveu d'ignorance est un label comme un autre). Le label est appliqué DES DEUX CÔTÉS : au catalogue central et au capteur (qui sait désormais résoudre localement ce régime : « référencé en local »).

I.8 La fédération entre sites

Fonction pure `mergeCatalogs` : agglomération gloutonne des centroïdes labellisés sous `link_thr` (0,06), label par vote majoritaire pondéré par les comptes (égalité : ordre lexicographique, pour le déterminisme), centroïde = moyenne pondérée re-normalisée, sites = union triée. Topologie supportée et documentée : la fédération se RECALCULE depuis les catalogues feuilles à chaque cycle (chaque feuille reste la source de vérité de son évidence) ; on ne re-fusionne pas un produit de fusion avec ses propres sources. Le scénario « un label appris au site A se résout au site B » est testé de bout en bout.

I.9 Le protocole capteur-central

Calqué sur l'API MCP du dispositif DriverV2 (JSON-RPC 2.0, push par notifications, pull par état), adapté au domaine industriel et câblé en mémoire pour cette version (le transport est hors périmètre, comme dans la spécification d'origine). Outils : `diagnostic_load_model`, `diagnostic_run`, `regime_current`, `catalog_apply_label`, `device_reset` (confirmation explicite exigée : code 1002 CONFIRM_REQUIRED sinon ; la portée « profiles » efface aussi la dernière fenêtre et la dernière empreinte : l'effacement est la promesse de premier rang), `capture_set_profile`. Notifications : `alarm` (NEW_REGIME, avec le centroïde du régime : dans ce domaine l'empreinte d'un régime moteur n'identifie personne, l'adaptation de la règle de confidentialité DriverV2 est documentée dans l'en-tête du contrat), `diagnostic_result`, `catalog_updated`, `status`. La table d'erreurs 1001-1005 de la spécification d'origine est reprise.

I.10 Les graphes de démonstration

Quatre fichiers `.spiky panda` (format v3 : layout + model + dashboards), versionnés dans le dépôt aux côtés des graphes existants, générés par un script (les indices de ports à la main ne survivent pas aux évolutions) :

- `motor-r385` : le BLOC MOTEUR seul, réutilisable : potentiomètre, onduleur, moteur R385, capteur de courant, scène. La charge est un nœud EXTERNE relié par un unique fil `tau_load` : changer de loi de charge = remplacer un nœud, sans toucher au bloc (exigence de réutilisation, prouvée par test).
- `motorwatch-r385` : le bloc + la chaîne de surveillance + le spectre/chute d'eau MCSA. Le nœud modèle est livré vide : on y dépose l'encodeur (entrée « `current_window` » [1,1,64], sortie « `embedding` ») et le graphe devient un moniteur vivant.
- `motor-induction` et `motorwatch-induction` : la même structure en triphasé (horloge + trois oscillateurs à 120 degrés, machine à cage, trois transducteurs, multiplexeur). Passer `brokenBars` à 2 en cours d'exécution fait monter les raies $f(1 \pm 2s)$ dans le spectre et lève NEW_REGIME.

Deux contraintes de câblage découvertes en faisant tourner les graphes

Le retour omega vers la loi de charge forme un CYCLE de flux : sur les moteurs à ports « `stream` » (induction), il DOIT passer par un canal de retour Z-1 (`Control.Feedback:channel`), exactement comme dans le graphe de référence ; le moteur CC, à ports « `signal` », tolère le fil direct. Tous les graphes livrés routent ce retour par le Feedback, en VUE SCINDÉE : ses deux demi-pavés (la moitié « entrée » près du moteur, la moitié « sortie » près de la charge) sont positionnés par le champ `anchors` du fichier, et aucun fil de boucle ne traverse le canevas.

La chaîne mono-canal doit passer par le multiplexeur même avec UNE seule entrée : c'est lui qui donne la dimension canal (`[T]` devient `[T,1]`) qu'exige la transposition vers `(1,1,T)`. Le port d'entrée du tampon de fenêtres est déclaré « `any` » : il ingère aussi bien des scalaires que des lignes tenseur du multiplexeur (une déclaration « `float` » ferait signaler le fil mux vers tampon comme incompatible par l'éditeur).

Limite assumée : l'encapsulation du bloc moteur dans un sous-graphe `Sim.Graph` reste un geste d'ÉDITEUR (le JSON interne sauvegardé n'est pas instancié au chargement par le runtime actuel) ; les graphes sont donc plats, et l'exigence de réutilisation est portée par le contrat « un seul fil de charge ».

PARTIE II : Journal de réalisation

Cette partie raconte le projet dans l'ordre où il s'est réellement déroulé, avec la même discipline que le cahier DriverV2 : chaque décision est motivée, chaque échec est documenté, et les chiffres cités sont ceux qui ont été mesurés (pas ceux qu'on espérait). Le projet s'est exécuté en une journée (11 juin 2026), en orchestrant des vagues d'agents de développement parallèles sur des périmètres de fichiers disjoints, chaque vague étant suivie d'une intégration et d'une vérification centralisées.

II.1 Cadrage : trois directives qui ont fait l'architecture

Le point de départ est la partie V.2.1 du cahier DriverV2 (« Industriel : régimes machine et diagnostic, la synergie la plus forte ») et un scénario exprimé par le donneur d'ordre : capteur de courant, découverte d'un régime nouveau, alarme, diagnostic différentiel par modèles ONNX téléversés au runtime, labellisation centrale, partage entre sites. Trois directives successives ont ensuite cadré l'architecture, chacune réorientant le plan en cours de route :

1. **Deux sous-projets** : d'abord un moteur R385 (courant continu classique), ensuite les brushless ET les machines à cage d'écureuil « pour simuler des cassures de barre » : le défaut MCSA de référence, documenté par les notes de recherche du dépôt.
2. **Tous les nœuds génériques en plugins** (en créer si nécessaire) pour être compatibles nodeeditor v2 d'emblée ; tout ajout 100 % générique de niveau framework dans core. L'application ne garde que le métier.
3. **Graphes versionnés et sérialisés**, éditables dans l'éditeur, avec la charge du moteur PILOTABLE INDÉPENDAMMENT : le graphe moteur doit resservir à d'autres applications.

L'exploration : ce qui existait, ce qui manquait

Un graphe d'exemple fourni par le donneur d'ordre (graph (28).json) a servi de pierre de Rosette : il contenait déjà le banc R385 complet (potentiomètre, onduleur PWM, moteur CC aux paramètres R385, capteur LEM, chaîne FFT, scène), et surtout il montrait le couple de charge composé À LA MAIN : omega, canal de retour Z-1, deux multiplications, K.omega2. Le manque nommé par le donneur d'ordre (« il manque la simulation du tau_load ») devenait précis : un nœud de PROFILS de charge.

L'inventaire systématique du dépôt (agents d'exploration parallèles) a établi les faits suivants :

- Aucun modèle de machine asynchrone nulle part (les seules occurrences de « cage » concernaient les cages de roulement) : le moteur à induction du sous-projet 2 serait une création.
- Aucun nœud de clustering, d'embedding ou de détection d'anomalie dans AUCUN plugin : le plugin ml serait une création.
- Le plugin DSP était riche (FFT, fenêtres, RMS, oscillateur, transducteur générique) mais sans détecteur de régime stationnaire, sans multiplexeur, sans fenêtres multi-canaux, sans transposition.
- Le moteur CC était devenu INTÉGRABLE (solveur RK4 de session, ports « signal ») alors que les brushless restaient en Euler-dans-fire à ports « stream » : deux régimes d'exécution à respecter.
- Le nœud modèle ONNX de l'éditeur savait charger un modèle par glisser-déposer et le ré-exécuter, mais PAS être câblé en flux dans un graphe (voir II.5).

II.2 Première vague : cinq chantiers de fondation en parallèle

Cinq agents ont travaillé simultanément sur des fichiers disjoints, les fichiers d'enregistrement partagés (index du plugin physics, manifestes, configurations racine) restant réservés à l'orchestrateur pour éviter tout conflit. Chaque chantier devait livrer ses tests verts avant intégration.

Chantier	Livré	Tests
Couple de charge	LoadTorqueNode, 5 profils, zéro allocation par tick	15
Nœuds DSP	porte de régime établi, multiplexeur, fenêtres [T,C], transposition	18
Plugin ml	bibliothèque de clustering portée de DriverV2 + nœud open-set + boilerplate complet du plugin	13
Téléversement validé	SHA-256 pur TypeScript (vecteurs FIPS) + loadModelValidated double banque	12
Machine à cage	modèle alpha-beta complet + asymétrie de barres + validation spectrale	15

Les pièges rencontrés et documentés pendant cette vague

- **La grammaire du bus d'alertes** : la sévérité « warning » n'existe pas ; le bus normalise tout ce qu'il ne connaît pas vers « info ». L'alarme du clusterer émet « warn », et un test fait transiter la trame par un vrai bus pour verrouiller la compatibilité.
- **Le protobuf trop poli** : des octets 0xFF en série « parsent » comme un modèle ONNX VIDE (fin de fichier immédiate) ; le chargeur validé rejette explicitement « modèle sans nœuds » pour interdire la bascule silencieuse vers un modèle blanc.
- **La calibration des barres cassées** : la proportionnalité raie/k/N n'est pas tombée du premier coup ; elle a été itérée jusqu'à mesurer, à $dt = 2e-4$: raie basse à 3,9 % du fondamental pour 2 barres sur 28, 7,5 % pour 4 (rapport 1,92). Ces chiffres seront ré-ancrés plus tard au nouveau taux déclaré (voir II.8).
- **Le manifeste en retard sur le code** : le manifest.json du plugin physics ne listait déjà plus tous les nœuds réellement enregistrés (onduleur et capteur de courant absents) : constat documenté, les ajouts du projet ont mis à jour code ET manifeste.

Intégration de la vague

Branchement des deux sous-plugins physics (load, motor-induction), création des alias d'espace de travail (tsconfig, jest), enregistrement du plugin ml partout où l'éditeur l'attend (script de déploiement des bundles, page node-editor-v2, balise script). Un seul accroc : le webpack du plugin onnx ne connaissait pas l'alias d'extension « .js vers .ts » (style d'import ESM des autres plugins) ; aligné sur la configuration du plugin physics. 77/77 tests verts après intégration, build TypeScript complet propre.

II.3 L'application : le métier, et un choix d'architecture défendu

L'application (catalogue, fédération, station, protocole, façade capteur) a été développée pendant que d'autres chantiers avançaient. Deux décisions notables :

Décision : le banc de diagnostic vit derrière un adaptateur, pas en nœud fractal nu

La session d'exécution fige la topologie du graphe à sa construction (états, capacités, entrées requises). Permuter les nœuds INTERNES d'un nœud modèle fractal en cours de session serait donc invisible pour la session en cours. La façade capteur tient le banc de diagnostic dans un petit kernel adaptateur : la frontière du graphe reste stable, loadModelValidated permute les banques DERRIÈRE elle. Le streaming fractal (II.5) sert le cas de l'éditeur ; l'adaptateur sert le cas du capteur toujours allumé.

Décision : le démarrage à froid est silencieux

Le premier profil découvert (la mise en service) ne lève PAS d'alarme : c'est la ligne de base. Chaque régime suivant alarme exactement une fois. Ce contrat est documenté dans la façade et asséré par les tests de bout en bout (3 régimes = 2 alarmes).

L'encodeur de test a demandé une vraie réflexion : le clusterer normalise les empreintes (distance cosinus), donc une simple mise à l'échelle du courant est INVISIBLE. L'encodeur synthétique (octets ONNX réels, générés par l'exportateur du dépôt : Flatten, Gemm, Relu, Gemm, à poids fixés) combine des bandes de niveau saturantes, une sensibilité de forme (|pentel) et un chemin de biais constant, de sorte que la DIRECTION de l'empreinte change avec le régime. Sur le banc R385 (7 V de moyenne PWM, trois régimes de charge), les distances cosinus inter-régimes mesurées valent 0,32 / 0,15 / 0,58 : toutes au-dessus du seuil d'assignation 0,05, avec un bruit intra-régime sous 1e-3.

II.4 La découverte qui a réorienté la semaine : le streaming fractal était cassé

Au moment d'écrire le graphe de démonstration, vérification de la mécanique d'insertion d'un modèle ONNX comme nœud : le routage d'entrée d'un graphe embarqué comparait le slot CÔTÉ SOURCE du canal (le nom de sortie du producteur amont) au lieu du slot de destination ; les canaux frontière que le mécanisme exigeait n'étaient créés nulle part ; et leurs constructeurs dans le builder étaient inutilisables (ils plantaient à la construction : cassés depuis leur création, jamais exercés). Autrement dit : la promesse « déposer un modèle et le câbler » de l'éditeur n'avait jamais pu fonctionner en flux.

Un chantier core dédié a établi le contrat de câblage par NOM de tenseur (toSlot côté entrée, fromSlot côté sortie), créé les canaux frontière au chargement du modèle, ajouté une session à valeur portée pour les ports pendants, et invalidé proprement les topologies internes au remplacement de modèle. Portes de régression larges : les suites onnx, dsp, ml, exécution, et les chaînes ONNX historiques (driverv2, cardriver) comme canaris du chemin infer(). Résultat inattendu et bienvenu : **7 tests préexistants du dépôt, en échec avant le projet, sont passés au vert** : la perte de jetons sur canaux pendants était leur cause racine commune.

II.5 Les graphes : deux leçons de câblage et un test phare

Les quatre graphes sont GÉNÉRÉS par un script Python (les indices de ports des connexions, à la main, ne survivent pas) puis validés par un chargeur headless qui reproduit exactement les règles de l'éditeur (création par registre, désérialisation des champs, canaux par noms, rattachement automatique du solveur via la scène). Premier passage : 5 tests sur 9.

- **Leçon 1, le cycle de charge** : omega vers la loi de charge vers tau_load vers le moteur est un CYCLE. Sur le moteur CC (ports « signal », écrasement sans file) le fil direct passe ; sur la machine à cage (ports « stream ») il fait déborder les canaux. Le graphe de référence avait montré la solution : un canal de retour Z-1. Les graphes triphasés l'utilisent.
- **Leçon 2, le canal fantôme** : la chaîne mono-canal envoyait des fenêtres [64] (rang 1) à une transposition qui exige le rang 2 ; le multiplexeur, même à une seule entrée, donne la dimension canal ([64,1]).

Après correction : 10 tests sur 10, dont le test phare du projet : charger motorwatch-r385.spikypanda tel quel, téléverser l'encodeur dans le nœud modèle DU GRAPHE CHARGÉ (le même chemin validé que la station), faire tourner : le profil de base se crée, puis un échelon de charge appliqué PAR LE SEUL FIL tau_load fait découvrir un second régime. Le fichier versionné est, littéralement, un moniteur open-set en boîte.

II.6 Sous-projet 2 : la cage d'écureuil de bout en bout

Le scénario industriel complet a été rejoué sur la machine à cage, avec un choix de traitement assumé : la façade capteur reçoit une ENVELOPPE démodulée du courant (RMS glissant sur EXACTEMENT une période d'alimentation, $dt = 1/(60 \times 84)$ pour annuler la porteuse), pas le 60 Hz brut. C'est la pratique MCSA standard, et c'est ce qui rend la signature lisible par l'encodeur de niveau : l'asymétrie de barres module l'enveloppe à 2.s.f (environ 10,6 Hz ici) avec une profondeur en k/N (mesurée : 4,2 % à sévérité 1, 8,0 % à sévérité 2).

1. Régime sain en charge (1,5 N.m, 80 V, 60 Hz) : ligne de base silencieuse.
2. Cassure de 2 barres déclenchée À CHAUD à $t = 5$ s, moteur en marche : EXACTEMENT une alarme NEW_REGIME.
3. Diagnostic différentiel poussé par la station (3 causes) : « broken_rotor_bar » gagne avec une marge de 3,1 ; le label atterrit au catalogue ET au capteur ; export, import au site B, fusion : compte sommé, sites « site-A,site-B », et le site B résout localement une empreinte de défaut voisine.
4. Contre-épreuve : un ÉCHELON DE CHARGE sain (1,5 vers 2,0 N.m, glissement 0,125 contre 0,088 pour le défaut) crée un troisième régime, diagnostiqué « load_change » : le modèle de diagnostic discrimine bien la cause, il ne crie pas « barre cassée » à chaque nouveauté.

Résultat de physique au passage : le PMSM en V/f boucle ouverte

Avec la résistance par défaut (0,5 ohm), le rotor à faible inertie POMPE et saute des pôles à la moindre perturbation (vitesse oscillant entre 56 et 233 rad/s) : le banc PMSM était inexploitable. $R = 2,0$ ohm amortit le mode oscillant (stabilité V/f classique) et les deux régimes se verrouillent exactement à 157,08 rad/s, rendant la signature purement électrique. Le test du sous-projet 2 documente ce réglage dans son en-tête.

II.7 Vérification globale et preuve de non-régression

- Suite complète du dépôt : 1 193 tests verts sur 1 198. Les 5 échecs restants ont été PROUVÉS antérieurs au projet : un arbre de travail propre sur le commit de départ (worktree dédié) reproduit exactement les mêmes 5 échecs (mpc, scénarios PMSM du paquet sensors, session de simulation, deux tests du scheduler dynamique et un du statique).
- L'éditeur vérifié au navigateur : les 9 plugins se chargent (dont le nouveau ml), 156 types de nœuds dans la palette, zéro erreur console.
- Construction TypeScript complète et bundles webpack de tous les paquets touchés (core, onnx, physics, dsp, ml, onnx-éditeur) déployés vers l'hôte.
- Un faux suspect : deux tests d'ENTRAÎNEMENT de réseaux (XOR, CNN) ont flanché pendant un passage global lancé en concurrence avec les agents : initialisation aléatoire non semée + contention CPU. Trois passages isolés verts, et le passage global propre les confirme : flakiness préexistante, pas une régression.

II.8 La revue contradictoire : neuf défauts réels que les tests rataient

Tout étant vert, une revue adversariale a été lancée : quatre dimensions (rayon d'impact core, physique, fidélité du portage clustering, contrats application/protocole), chaque finding contre-vérifié par un agent chargé de le RÉFUTER, reproduction exécutée à l'appui. Un réviseur est mort en vol (erreur réseau) : sa dimension a été rejouée par reprise du workflow (les résultats déjà acquis étant servis depuis le cache). Bilan : 9 défauts confirmés (3 majeurs, 6 moyens), plusieurs allégations réfutées avec preuves, et une grappe de défauts mineurs. Tous les confirmés ont été corrigés selon la même discipline : test de

reproduction d'abord (échec constaté), correctif ensuite, portes de régression larges enfin. Le détail complet est en annexe B ; les trois majeurs et deux leçons méritent le corps du texte :

- **Fan-out de sortie d'un modèle embarqué** (majeur) : la valeur d'un port de sortie était CONSOMMÉE par le premier canal parent ; tout deuxième consommateur (modèle vers détecteur ET courbe, un câblage d'éditeur banal) était affamé en silence. Correctif : lire une fois, publier à tous les canaux concordants, consommer une fois.
- **Modèle partiellement câblé** (majeur) : à moitié alimenté, le graphe interne débordait au deuxième tick et l'exception TUAIT la session parente, avec un message désignant un slot interne que l'utilisateur n'a jamais câblé. Correctif : exécution « tout ou rien » sur les ports déclarés ; les jetons orphelins sont jetés, la session vit.
- **Le déclencheur de re-clustering en port de données** (majeur) : le runtime compte TOUT port de données câblé comme requis (l'attribut « optional » est mort à l'exécution) ; câbler le déclencheur gelait donc le nœud puis faisait déborder le canal d'empreintes. Correctif : passage au plan de contrôle (« _recluster », drainé par processControllInputs), conformément à la convention du plugin logic ; et la sortie k n'est plus publiée qu'une fois par tir.

Leçon 1 : une revue qui s'exécute vaut dix relectures

Chaque finding confirmé l'a été par REPRODUCTION (test jeté ou script), pas par plausibilité. Plusieurs allégations bien argumentées ont été réfutées de la même manière : la « fuite de confidentialité » du centroïde dans les alarmes est un contrat adapté, documenté et testé (un régime moteur n'identifie personne) ; le « double comptage » de la fusion n'existe pas dans la topologie supportée (recalcul depuis les feuilles).

Leçon 2 : le correctif qui casse plus loin que ses portes

Le correctif « pendingInput consommé à la lecture » (contre le mélange de tenseurs frais et périmés) a cassé l'entraînement CNN du dépôt : les boucles d'entraînement injectent UNE fois et relisent à chaque passe. Les portes de régression du chantier ne couvraient pas ces consommateurs lointains. Sémantique finale : PERSISTANCE sur les nœuds source purs (aucun canal entrant : le mélange y est impossible), consommation à la lecture uniquement sur la branche frontière. Moralité : pour un changement de core, la porte de régression minimale est la suite COMPLÈTE du dépôt.

Côté physique, deux corrections de fond issues de la même revue : la loi quadratique de charge devait être SIGNÉE ($k \cdot \omega \cdot |\omega|$) ; la version $k \cdot \omega^2$ propulsait un moteur en rotation inverse jusqu'au NaN, reproduit numériquement ; et le taux d'échantillonnage auto-déclaré de la machine à cage ignorait la résistance rotorique (glissement faussé de moitié au taux déclaré, divergence d'Euler dès R_r supérieur à 19 fois R_s , reproduites toutes deux). Le taux est désormais calé sur le vrai pôle rapide et les calibrations de l'en-tête sont RÉ-ANCRÉES au taux déclaré (~9,8 kHz) : raies à 3,5 % (2/28) et 6,7 % (4/28), glissement 9,2 % contre 10,1 % pour la référence sur-échantillonnée x4.

Côté application, deux moyens : une trame pouvait chevaucher un changement de régime via la fenêtre d'hystérésis de la porte (3 % des transitions environ : régime fantôme reproduit ; corrigé par une quarantaine de breakHold-1 échantillons dans la façade, la porte et son contrat restant intacts) ; et un modèle de diagnostic au mauvais nombre de scores passait la validation puis échouait APRÈS la bascule (demi-engagement ; corrigé par la validation de FORME de sortie avant bascule, plus un chemin d'exécution qui distingue « chargement réussi, exécution échouée »).

II.9 État final

Vérification	Résultat
Suites du périmètre motorwatch (15 suites)	139 / 139 verts
Dépôt complet	1 193 / 1 198 (5 échecs prouvés antérieurs, 7 antérieurs réparés)
Construction TypeScript (tsc -b, toutes références)	propre
Bundles webpack + déploiement hôte	tous les paquets touchés, propres
Éditeur nodeeditor v2 au navigateur	9 plugins, 156 types, 0 erreur console
Lint / format sur les fichiers touchés	propres (eslint + prettier)

II.10 Retour d'usage immédiat : sept défauts signalés aux premiers essais

Les premiers chargements et les premières exécutions des graphes par le donneur d'ordre ont remonté sept anomalies, toutes réelles, toutes corrigées dans la foulée (avec la réponse à la question légitime « comment cela peut-il fonctionner ? » : le runtime ne valide JAMAIS les types de ports, qui sont des métadonnées d'interface ; seul le rendu de l'éditeur signale l'incohérence, et les tests headless activaient eux-mêmes les plugins manquants ou tournaient à une cadence qui masquait le défaut).

- **Fil « tensor vers float » signalé invalide** (multiplexeur vers tampon de fenêtres) : le port d'entrée du tampon déclarait encore « float » alors que le nœud avait appris les lignes tenseur. Le type déclaré a été élargi à « any » dans le nœud ET dans son enregistrement ; les graphes sont régénérés en conséquence.
- **Le Feedback chargé en boîte unique, fils de boucle dessinés** : la cause était une divergence entre deux chemins de l'éditeur. Au dépôt depuis la palette, la définition visuelle d'un nœud est construite depuis le MÉTA du registre (nombre d'ancres de la vue scindée, routage des ports par ancre, ports de contrôle, descripteurs variadiques) ; au CHARGEMENT d'un fichier, elle était reconstruite depuis les seules listes de ports sauvegardées, qui ne portent rien de tout cela. Correctif générique dans l'éditeur : un enrichissement de la définition par le méta (fonction pure, testée sans DOM), qui préserve l'ORDRE des ports du fichier (les connexions s'y réfèrent par indice, et les ports variadiques poussés n'existent que là) tout en faisant revenir ancres, types faisant autorité, contrôles et variadiques. Vérifié au navigateur sur les graphes livrés : Feedback en deux demi-pavés aux positions sauvegardées, zéro lien incompatible, et le port libre suivant du multiplexeur (in_3) poussé au chargement comme au dépôt.
- **Plantage au Play (« this._ofin.opsc is not a function »)** : la page node-editor-v2 ne chargeait PAS le plugin ONNX (jamais référencé : ni balise script, ni appel de chargement), donc « spk.onnx:model » manquait au registre, le nœud encodeur se chargeait en « layout-only » (sa donnée restait le blob JSON brut du fichier) et le constructeur de canal explosait au câblage de la session. Double correctif : le plugin ONNX est désormais chargé par la page (le nœud modèle et Conv rejoignent la palette), ET le constructeur de session ignore proprement, avec avertissement, tout fil dont une extrémité est layout-only (le même duck-typing que la collecte des nœuds : sans fire(), pas de câblage) : un typeld manquant ne tue plus jamais le Play, il prive seulement son nœud de ses fils. Vérifié au navigateur : bootstrap du Play sans erreur, moteur intégré en pas-à-pas (i = 0,66 A après 40 pas).
- **Les nœuds Start / Stop perdus à la désérialisation** : l'intuition « la désérialisation ne reconstruit pas un nœud correct » a été vérifiée champ par champ : les valeurs des nœuds TYPÉS revenaient toutes exactes (moteur, potentiomètre, charge, porte, clusterer). Le vrai coupable était structurel : l'hôte créait les nœuds de cycle de vie Start / Stop SANS typeld ; toute sauvegarde les réduisait donc à des blobs JSON au rechargement, Session.start() ne trouvait plus de StartNode à armer, et

tout câblage « Start vers déclencheur » mourait en silence. Correctif générique : les nœuds de cycle de vie sont désormais enregistrés comme tout le monde (Core.Lifecycle:start / :stop), l'hôte les crée via le registre, et les graphes livrés les typent. Vérifié au navigateur : après save puis load, Start et Stop reviennent en vraies instances (fire et arm présents) ; le seul nœud non runtime restant est la scène Earth, par conception (GraphItem consommé par liens de configuration).

- **La rampe de charge non bornée** : le profil « ramp » montait pour toujours ($\tau_0 + \text{pente} \times t$), dépassait le couple de décrochage du R385 (~45 mN.m à 6,7 V) et surtout ne se STABILISAIT jamais : aucun régime établi, donc aucune fenêtre, donc aucune alarme possible. Nouvelle sémantique : la rampe dérive de τ_0 VERS τ_1 puis tient le palier (la dérive lente VERS un nouveau régime, que la porte peut capturer).
- **Le papillotement de la porte à la cadence dérivée** : à 200 kHz (20 x fPwm exigés par l'onduleur), l'ondulation PWM est résolue à $\pm 6\%$ alors que la tolérance de la porte est de 5 % : la porte n'accumulait jamais ses échantillons stables et la chaîne restait muette : la SECONDE cause du « jamais d'alarme ». Les tests headless tournaient à 10 kHz, où le PWM s'échantillonne en stroboscope : ils ne pouvaient pas le voir (leçon : la cadence est un paramètre de test). Correctif générique : un lissage EMA optionnel de la DÉCISION de stabilité (smoothAlpha, défaut 1 = inchangé) ; value_gated continue de transmettre les échantillons BRUTS à l'encodeur. Le graphe livré règle smoothAlpha 0,01 et des temporisations adaptées. Vérifié dans l'éditeur : l'alarme NEW_REGIME (sévérité warn) atteint le bus d'alertes.
- **Les booléens invisibles du panneau de propriétés** : « je n'ai pas yAuto » : exact, et le champ était pourtant bien déclaré éditable ; c'est le panneau qui ne savait pas rendre les BOOLÉENS (les widgets intégrés couvraient nombre, chaîne et vecteurs). Un éditeur case à cocher générique a été ajouté : yAuto, dbScale, add_batch_dim et tous les booléens du système apparaissent désormais. Au passage, le cache-buster des bundles (?v=p41), jamais incrémenté pendant les correctifs du jour, a été bumpé : un navigateur au cache tiède pouvait servir d'anciens bundles et brouiller le diagnostic.

Deux retouches d'ergonomie ont immédiatement suivi l'apparition de la case yAuto (le réglage, invisible jusque-là, n'avait jamais été exercé) : au passage en échelle manuelle, le tracé sautait sur les bornes par défaut [-1, 1] et disparaissait si le signal en sortait ; le plot capture désormais l'ÉCHELLE VISIBLE courante comme bornes (sémantique oscilloscope, vérifié au navigateur : bascule à [-0,30, 3,10] sans saut). Et le nœud Print String, fidèle à la sémantique UE5 (déclencheur sur in, contenu sur text), affichait une ligne vide quand on lui câblait directement une charge utile : il imprime désormais la charge non-trigger arrivée sur in, et un jeton sur text seul suffit à déclencher (mode données), sans casser le mode exec historique (4 tests).

II.11 La contrainte d'acquisition : le capteur reprend son horloge

Le diagnostic du papillotement (II.10) a fait émerger chez le donneur d'ordre une contrainte de conception plus profonde : **les capteurs ont leur propre capacité d'échantillonnage**, indépendante du pas de la simulation. Profil imposé (référéncé IEC 61430 / 61407) : 10,24 kHz, blocs de 2 048 échantillons, 200 ms. Le portage de cette contrainte a remplacé le contournement par la bonne architecture : un nœud d'acquisition générique (DSP.Acquire:daq) dont l'horloge dérive du temps simulé (mêmes blocs que la session tourne à 20 kHz ou à 200 kHz : la propriété est testée), avec anti-repliement, RMS par bloc, et rafales bornées. Bénéfices immédiats mesurés : la porte de régime travaille au RMS par bloc (le lissage au niveau échantillon devient inutile dans le graphe livré) ; la FFT passe à 2 048 points et gagne la résolution de 5 Hz qui sépare réellement les bandes latérales MCSA ; et le scénario complet a été rejoué dans l'éditeur au profil par défaut : 16 blocs de référence (RMS 0,654 A), échelon de charge par l'unique

fil tau_load, RMS 1,616 A, nouveau régime découvert et alarme NEW_REGIME sur le bus. Le re-fenêtrage bloc vers fenêtre encodeur utilise les nœuds existants (Frame 64 / hop 2048 : le cliché de tête de bloc), et le risque résiduel de fenêtre à cheval sur une transition tombe à $64/2048 = 3 \%$ par changement de régime (la quarantaine de la façade capteur le couvre côté MCU).

Lacune d'éditeur notée au passage (annexe D) : la sauvegarde ne sait pas sérialiser les connexions de ports de CONTRÔLE (l'index est cherché dans les ports de données) ; les graphes livrés n'en utilisent donc pas.

II.12 La grenouille bouillie : l'ancre de référence

Dernier retour du donneur d'ordre, et le plus profond : en mode rampe de charge, AUCUN changement de régime n'est détecté. Son diagnostic était déjà la solution : **« une fois qu'une référence est acquise, on doit la garder comme un snapshot, sinon un dérèglement progressif dans le temps ne sera jamais détecté »**. L'analyse du code a confirmé le mécanisme exact : quand chaque pas de la rampe reste sous update_thr (0,02), le centroïde du profil SUIT le signal par lissage EMA ; la référence marche AVEC la dérive, la distance d'assignation ne croît jamais, et une machine peut s'user jusqu'à sortir complètement de sa ligne de base sans une seule alarme. C'est le problème de la grenouille bouillie. Le seuillage update_thr hérité de DriverV2 était le compromis anti-dérive pour des CONDUCTEURS, dont le profil comportemental ne doit pas ramper ; une machine industrielle, elle, DÉRIVE par conception (l'usure), et le suivi seul est un angle mort.

Le correctif suit l'intuition à la lettre : chaque profil garde désormais une ANCRE, cliché immuable de son centroïde pris à la création. À chaque assignation, le centroïde de suivi est comparé à l'ancre ; passé drift_thr (défaut 0,1, soit $2 \times \text{assign_thr}$: une référence qui a silencieusement marché deux fois le rayon de l'ensemble ouvert n'est plus la référence apprise), le nœud publie UNE alarme REGIME_DRIFT (sévérité warn, charge utile portant driftSteps) et RÉ-ANCRE le profil sur son centroïde courant. Sémantique d'escalier : chaque alarme signifie « la référence a bougé d'encore drift_thr depuis la dernière ancre » ; une dérive continue produit un train régulier d'alarmes au lieu du silence, un régime stable n'en produit aucune. Le recalcul par lots (_recluster) reste un RE-BASELINING délibéré : profils reconstruits, ancres fraîches, compteurs remis à zéro ; et drift_thr 0 désactive tout (l'ancien comportement, conservé comme épingle de régression).

Une correction de géométrie mérite la trace : la distance cosinus est QUADRATIQUE en angle, donc le nombre de marches d'une dérive angulaire constante suit la longueur d'ARC ($\text{floor}(\text{angle total} / \arccos(1 - \text{drift_thr}))$), pas le quotient naïf distance totale / drift_thr, géométriquement inatteignable ; les tests épinglent l'attendu correct à plus ou moins une marche.

Côté protocole, le code d'alarme du fil s'élargit à « NEW_REGIME | REGIME_DRIFT » ; la façade capteur transmet la dérive SANS suppression de démarrage à froid (une dérive lente du régime de base commence légitimement à $k = 1$, c'est précisément le signal pour lequel l'ancre existe). Validation : 60 / 60 tests verts, dont la nouvelle suite de dérive (épingle de régression drift_thr 0 sur 500 pas de rotation lente, escalier de 4 marches sur une rotation de $2\pi/3$ en Session réelle, échelon net sans fausse dérive, 4 000 pas de gigue bénigne sans alarme, ré-ancrage au recluster, traversée du bus d'alertes) et un test bout-en-bout : rampe d'enveloppe 0,3 vers 1,4 A sur ~620 fenêtres, zéro NEW_REGIME (suppression à froid intacte), escalier REGIME_DRIFT croissant jusqu'au flux de notifications du serveur.

Et la preuve a été rejouée en direct dans l'éditeur, sur le scénario exact du rapport d'origine : graphe R385 livré, encodeur chargé et validé (sha256), profil de charge en rampe LENTE. Pendant seize secondes simulées de dérive, zéro création de profil (l'angle mort d'avant, reproduit) ; à la marche 0,1, l'alarme « REGIME_DRIFT label=2 step=1 distance=0.1006 » est apparue sur le bus d'alertes et le ré-

ancrage s'est observé en direct (distance à l'ancre retombée de 0,0918 à 0,0001). Les graphes livrés exposent drift_thr (0,1) en éditable et drift_count en lecture.

Note de méthode : aucun commit n'a été créé pendant le projet ; l'arbre de travail porte l'intégralité du livrable, et ce cahier est généré depuis des sources versionnées à ses côtés.

PARTIE III : Guide de reconstruction

Cette partie donne les chemins, les commandes et les contrats nécessaires pour reconstruire, vérifier et étendre le système. Convention du dépôt : toutes les commandes se lancent depuis la RACINE de l'espace de travail (jamais de cd dans les paquets).

III.1 Où vivent les fichiers

Zone	Contenu
packages/dev/plugins/physics/src/mechanical/load/	LoadTorqueNode + enregistrement du sous-plugin
packages/dev/plugins/physics/src/electric/motor-induction/	InductionMotorDynamicNode + enregistrement
packages/dev/plugins/dsp/src/{detect,stream,tensor}/	porte de régime établi ; multiplexeur + tampon [T,C] ; transposition
packages/dev/plugins/ml/	plugin complet : src/cluster/clustering.ts (bibliothèque), online-cluster.node.ts, manifest, webpack
packages/dev/plugins/onnx/src/graphs/	model.graph.ts (loadModelValidated, ports frontière), sha256.ts
packages/dev/core/src/execution/, .../compute/	routage fractal (execution.graph.ts), builder, kernel (pendingInput)
packages/dev/applications/motorwatch/src/	central/ (catalog, federation, station), edge/ (device, adaptateur), protocol/mcp.ts
packages/host/www/data/graphs/*.spikypanda	les 4 graphes de démonstration (v3)
scripts/generate-motorwatch-graphs.py	le générateur des graphes (à relancer après tout changement de ports)
packages/tests/{motorwatch,ml,dsp,onnx-plugin,physics,execution}/	les 15 suites du périmètre

III.2 Construire et vérifier

```

npm run build:dev                # tsc -b, toutes references
npx jest packages/tests/motorwatch # central, r385, pmsm, induction, bord de regime, roundtrip graphes
npx jest packages/tests/ml packages/tests/dsp packages/tests/onnx-plugin
npx jest packages/tests/physics/loadtorque.node.test.ts packages/tests/physics/induction-motor.test.ts
npx jest packages/tests/execution/boundary-ports.test.ts
npx jest                         # depot complet (5 echecs preexistants attendus)
python scripts/generate-motorwatch-graphs.py # regenere les 4 .spikypanda
npm run bundle -w @spikypanda/plugin-ml      # idem physics/dsp/onnx si touches
node scripts/deploy-bundles.mjs              # copie vers packages/host/www/bundle
npm run server                               # puis /node-editor-v2/index.html

```

Les cinq échecs préexistants

mpc (résolution de module), sensors/pmsm-scenarios, sim/sim-session, execution/dynamic (2 tests) et execution/static (1 test) échouent AVANT le projet : la preuve a été faite sur un arbre de travail propre du commit de départ. Toute autre rougeur est une régression à traiter.

III.3 Les contrats d'interface

Nœuds livrés (entrées, sorties, réglages clés)

Nœud	Entrées	Sorties	Réglages
Load:torque	omega (opt.)	tau_load	profile, tau0/tau1, tStep, rampRate, k, amplitude, frequency
Induction:dynamic	V_a V_b V_c, tau_load, dt (opt.)	i_a i_b i_c, omega, theta_m, tau_em, slip	Rs Rr Ls Lr Lm P J b, f_supply, broken_bars, total_bars, bar_severity
Detect:steadystate	value	value_gated, steady, transition	epsilon, settle, breakHold, emaAlpha
Stream:mux	in_0..in_N (variadique)	frame [N]	
Stream:buffer	value (scalaire ou [C])	frame [T] ou [T,C]	frameSize, hopLength
Tensor:transpose	tensor [A,B]	transposed [B,A] ou [1,B,A]	add_batch_dim
ML.Cluster:online	embedding ; _recluster (contrôle)	label, is_new, distance, k, alarm	assign_thr, update_thr, alpha, link_thr, k_max, history_max

Câblage d'un modèle ONNX en flux (contrat core)

- Vers le modèle : le toSlot du canal est le NOM du tenseur d'entrée ONNX. Exécution « tout ou rien » : chaque entrée déclarée doit recevoir un jeton au même tick, sinon les jetons sont jetés et l'exécution est sautée.
- Depuis le modèle : le fromSlot du canal est le NOM du tenseur de sortie ; la valeur est diffusée à tous les canaux parents demandeurs.
- loadModelValidated(bytes, { name, sha256, expectInputShape, expectOutputCount, expectOutputShape }) renvoie { ok, error, sha256, inputNames, outputNames } ; tout échec laisse l'ancien modèle ET ses ports intacts (dimensions <= 0 = jokers).

Le protocole capteur-central (in-process)

```

outils      : diagnostic_load_model { model_bytes, sha256?, causes[] }
              diagnostic_run { causes? }          (re-exécution sans transfert)
              regime_current {}
              catalog_apply_label { label, centroid }
              device_reset { scope: profiles|all, confirm: true }  (1002 sinon)
              capture_set_profile { preset | seuils }
notifs      : alarm (NEW_REGIME, detail { k, label, distance, centroid })
              diagnostic_result, catalog_updated, status
erreurs     : 1001 BUSY, 1002 CONFIRM_REQUIRED, 1003 INVALID_STATE,
              1004 PERSIST_FAILED, 1005 DIAGNOSTICS_OFF (table héritée de DriverV2)

```

III.4 Synthétiser les modèles ONNX de test et de démonstration

Les modèles des TESTS sont générés à poids fixés, en mémoire, par l'exportateur public du dépôt (OnnxGraphExporter), selon le patron de packages/tests/motorwatch/helpers.ts. Les deux artefacts de DÉMONSTRATION référencés par les graphes de l'éditeur sont exportés sur disque par la suite demo-models.export.test.ts vers packages/host/www/data/models/ : motorwatch-encoder-demo.onnx (à déposer sur le nœud modèle de motorwatch-r385) et motorwatch-diagnostic-demo.onnx (la tête différentielle à 3 causes). Les octets sont déterministes : supprimer les fichiers est sans risque, un passage de la suite les régénère à l'identique.

- **Encodeur** : entrée « current_window » (1, 1, T), sortie « embedding » (1, 5). Topologie DriverV2 : Conv1d(1 vers 8, noyau 5) + Relu, deux étages Conv1d(8 vers 8, noyau 3, identité au tap central) + Relu, GlobalAveragePool, Flatten, Gemm(8 vers 5) ; 493 paramètres fixés à la main. Le premier étage calcule les bandes de niveau saturantes (noyaux moyenneurs, biais -0,15 / -0,50 / -0,85 / -

1,20), la paire de dérivées centrées ($|pente|$) et un canal constant 0,2 (noyau nul, biais) ; la tête recompose les bosses de bande (gains $1/2/3$), la pente (avec un terme correcteur $-5/(T-1,2)$ fois la moyenne brute, qui annule EXACTEMENT la fuite continue du zero-padding dans les canaux de dérivée) et le biais. Le biais est ESSENTIEL : le clusterer normalise, seule la direction compte.

- **Diagnostic** : un seul Gemm de l'empreinte normalisée vers un score par cause ; les poids se choisissent pour donner une marge nette à la cause attendue et un score NÉGATIF aux contre-épreuves.

III.5 Les calibrations qui comptent

Grandeur	Valeur et provenance
R385	R 1,22 ; L $1e-3$; $K_e = K_t 8,22e-3$; J $6e-7$; b $1,03e-6$ (graphe de référence du banc physique)
Machine à cage (défauts)	$R_s 2,3$; $R_r 2,5$; $L_s = L_r 0,23$; $L_m 0,22$; P 2 ; J $5e-3$; b $1e-4$; 60 Hz
Banc induction	80 V crête, 1,5 N.m, $dt = 1/(60 \times 84)$ (la fenêtre RMS annule exactement la porteuse)
Taux requis induction	$\tau_e = D/(R_s L_r + R_r L_s)$, $40/\tau_e$, plancher 80.f : ~9 813 Hz aux défauts
Raies de barres (au taux déclaré)	2/28 : basse 3,5 %, haute 0,28 % ; 4/28 : 6,7 % / 0,50 % ; glissement 9,2 %
Clustering	assign 0,05 ; update 0,02 ; alpha 0,15 ; link 0,06 ; k_{max} 4 ; historique 512
Acquisition (DSP.Acquire:daq)	fs 10,24 kHz ($2,56 \times F_{max}$, F_{max} 4 kHz) ; blocs 2 048 éch. = 200 ms ; résolution FFT 5 Hz ; anti-repliement 1 pôle à F_{max} ; RMS par bloc en sortie
Porte (graphes, cadence bloc)	epsilon 0,05 ; settle 5 blocs ; breakHold 2 ; emaAlpha 0,2 ; smoothAlpha 1 (inutile à 5 Hz)
Porte + quarantaine (façade capteur, cadence échantillon)	epsilon 0,05 ; settle 20-40 ; breakHold 3 ; smoothAlpha 0,005-0,02 si porteuse résolue ; quarantaine breakHold-1 échantillons
Diagnostic	marge de labellisation 0,25 ; en deçà : « regime_k_unlabeled »

III.6 Étendre le système

- **Ajouter un type de moteur** : suivre le patron motor-induction (classe FaultableNode + IDependsPorts, computeRequiredHz calé sur le pôle rapide RÉEL du modèle, enregistrement dans le sous-plugin + index + manifeste du plugin physics) ; valider par un test spectral, pas seulement par des invariants.
- **Ajouter un profil de charge** : un cas dans LoadTorqueNode + sa loi dans le commutateur ; respecter la convention de passivité ($\tau.\omega \geq 0$ pour les profils dépendant de la vitesse).
- **Ajouter une cause de diagnostic** : étendre la liste de causes côté station ET la dimension de sortie du modèle ; expectOutputShape garantit la cohérence à la bascule.
- **Brancher un transport réel** : implémenter IDependsServer au-dessus de MQTT/BLE ; le contrat in-process est la spécification ; le découpage en trames du téléversement (chunks + reprise) est l'extension notée « hors périmètre » de l'API DriverV2, à traiter au même endroit.
- **Régénérer les graphes** : TOUT changement de ports ou de champs sérialisés des nœuds impose de relancer le générateur puis la suite graphs.roundtrip (c'est elle qui garantit que les fichiers versionnés restent chargeables dans l'éditeur).

ANNEXES

Annexe A : correspondance brique à brique avec le cahier DriverV2

Le cahier DriverV2 (partie V.1) recensait quatre briques réutilisables et une méthodologie. Voici leur incarnation exacte dans motorwatch :

Brique DriverV2	Incarnation motorwatch	Ce qui a changé
Brique 1 : invariance au montage (repère véhicule)	sans objet direct : le courant n'a pas d'orientation	remplacée par la normalisation d'enveloppe (référence de mise en service)
Brique 2 : acquisition événementielle (manœuvres)	DSP.Detect:steadystate + quarantaine	sémantique INVERSÉE : on capture les états établis, pas les transitoires ; la machine à états à hystérésis est la même
Brique 3 : encodeur ~20 Ko, boîte noire ONNX	contrat identique (fenêtre vers empreinte normalisée)	l'encodeur livré est synthétique à poids fixés ; l'entraînement contrastif sur simulateur est l'étape V.3 à rejouer
Brique 4 : clustering anonyme à k émergent	plugin ml : portage ligne à ligne (assignation en ligne + agglomératif à seuil absolu)	historique borné en anneau ; déclencheur de re-clustering déplacé sur le plan de contrôle ; alarme câblable au bus d'alertes
Méthodologie : sim vers réel, négatifs documentés	journal partie II ; revue contradictoire avec reproductions	ajout : la revue adversariale multi-agents comme étape de sortie

S'y ajoutent les pièces NOUVELLES propres au scénario industriel : le téléversement de modèles au runtime (que l'API DriverV2 notait explicitement « hors périmètre, canal OTA à part »), le diagnostic différentiel, le catalogue labellisé et la fédération entre sites.

Annexe B : les neuf défauts confirmés par la revue, et leurs correctifs

Défaut (sévérité)	Où	Correctif
Fan-out de sortie d'un modèle embarqué : seul le premier consommateur servi (MAJEUR)	core / routage fractal	lecture unique par port, publication à tous les canaux concordants, consommation unique
Modèle partiellement câblé : débordement interne au tick 2, session parente tuée (MAJEUR)	core / routage fractal	exécution « tout ou rien » sur les ports déclarés ; jetons orphelins jetés
Déclencheur de re-clustering en port de données : gel puis débordement (MAJEUR)	plugin ml / nœud clusterer	port de plan de contrôle « _recluster » + drapeau drainé ; k publié une seule fois par tir
pendingInput périmé mélangé aux jetons frais (MOYEN)	core / kernel	persistance sur source pure, consommation à la lecture sur la branche frontière (après un premier correctif trop large qui cassait l'entraînement CNN)
Loi quadratique de charge non passive : NaN en rotation inverse (MOYEN)	plugin physics / LoadTorque	loi signée $k.\omega.\lvert\omega\rvert$ + borne « ne jamais entraîner »
Taux requis de la machine à cage aveugle à Rr : glissement faussé de moitié, divergence Euler (MOYEN)	plugin physics / induction	vrai pôle rapide $D/(R_s.L_r + R_r.L_s)$, marge 40/tau, plancher 80.f ; calibrations ré-ancrées

Défaut (sévérité)	Où	Correctif
Trame chevauchant un changement de régime via la fuite d'hystérésis (~3 % des transitions) (MOYEN)	application / façade capteur	quarantaine de breakHold-1 échantillons, vidée avant la remise à zéro du tampon ; la porte et son contrat restent intacts
Téléversement de diagnostic à demi engagé sur mauvais nombre de scores (MOYEN)	plugin onnx + application	validation de FORME de sortie avant bascule ; « chargé mais non exécuté » distinct d'un rejet
k publié deux fois dans un même tir (assignation + re-clustering) (MOYEN)	plugin ml / nœud clusterer	publication unique en fin de tir, valeur finale

Défauts mineurs corrigés dans la foulée : plage du glissement étendue à $[-1, 2]$ (freinage en contre-courant) ; ingestion d'empreintes tolérante à tout tableau numérique (un tenseur Float64 était jeté en silence) ; restriction du repli de routage hérité (jeton mal aiguillé vers un port homonyme après remplacement de modèle) ; compteur de trames avalées exposé dans l'état du capteur ; l'effacement « profiles » purge aussi la dernière fenêtre et la dernière empreinte ; historique de la station en anneau borné + déduplication des téléversements par SHA (outil diagnostic_run sans transfert).

Annexe C : allégations réfutées (et pourquoi c'est important)

- **« Le centroïde dans les alarmes viole la règle de confidentialité DriverV2 »** : réfuté. La règle DriverV2 protège des EMPREINTES DE PERSONNES (un embedding de conduite identifie un humain) ; un centroïde de régime moteur n'identifie personne. L'adaptation est documentée dans l'en-tête du contrat et le chemin rapide du catalogue en dépend ; trois tests l'assèrent.
- **« La fusion de catalogues double-compte l'évidence en cas de re-fédération »** : réfuté pour la topologie supportée. La fusion est une fonction pure recalculée depuis les catalogues FEUILLES ; nul ne re-fusionne un produit de fusion avec ses sources. Le mode d'emploi (une phrase) suffit ; la propriété mathématique sous-jacente (l'agglomération sur centroïdes n'est pas idempotente) est réelle mais inatteignable ici.
- **« Le re-clustering efface des profils dont l'évidence a quitté l'historique borné »** : mécaniquement exact, mais inatteignable sur le capteur livré (rien n'y déclenche de re-clustering) et couvert par le contrat documenté « les étiquettes locales ne sont jamais stables à travers un re-clustering ». Conservé en annexe D comme piège latent.

La valeur de cette annexe : la revue n'a pas seulement trouvé des défauts, elle a aussi établi par preuve ce qui N'EST PAS un défaut, et ces justifications sont désormais opposables aux relectures futures.

Annexe D : limites connues et travaux futurs

- **Encodeur entraîné** : la boucle « simulateur statistiquement réaliste, entraînement contrastif, transfert gelé mesuré sur réel » (DriverV2 partie V.3) reste à rejouer pour le courant moteur ; le jeu public UFU (barres cassées) est le candidat naturel pour la campagne de transfert.
- **Re-clustering périodique** : le déclencheur existe (plan de contrôle) mais n'est exposé ni par un outil du protocole ni par la façade ; sur un capteur allumé des mois, c'est le mécanisme anti-sur-segmentation (et la liste de centroïdes n'est pas bornée, contrairement à l'historique).
- **Dérive lente des références de labels** : les centroïdes du clusterer suivent le régime (EMA) tandis que les références de labels et le catalogue gardent le centroïde de découverte ; un régime qui dérive lentement peut rester assigné mais perdre son label local. Piste : rafraîchir la référence au rythme de l'assignation.

- **Rafales sur les ports de sortie fractals** : la session à valeur portée garde la DERNIÈRE valeur par port et par tir ; un sous-graphe qui émettrait plusieurs jetons sur un même port de sortie en un tir n'en livrerait qu'un (documenté ; file bornée si le besoin apparaît).
- **Producteurs ONNX multi-sorties** : l'appariement sortie-canal est positionnel dans le kernel ; exact pour les modèles à sorties uniques (la totalité du registre actuel), à indexer le jour où un opérateur multi-sorties réel arrive.
- **Sim.Graph au chargement** : le JSON interne d'un sous-graphe sauvegardé n'est pas instancié par le runtime au chargement d'un fichier ; le jour où il l'est, l'encapsulation du bloc moteur redevient la forme naturelle des graphes de démonstration.
- **Connexions de ports de contrôle non sérialisables** : la sauvegarde de l'éditeur cherche l'index du port dans les tableaux de ports de DONNÉES (assertion non nulle) ; un fil branché sur _enable / _started ferait échouer save(). Les graphes livrés n'utilisent aucun fil de contrôle ; à corriger le jour où le câblage Start vers déclencheurs doit survivre à un cycle save / load.
- **Transport et OTA réels, portage MCU** : voir partie III.6 ; rien dans l'architecture ne s'y oppose, rien n'en est fait.

Annexe E : glossaire

Terme	Définition
Ensemble ouvert (open-set)	régime de fonctionnement où le système découvre lui-même ses catégories, sans liste fixée à l'avance ; « combien de régimes ? » est une SORTIE, pas une entrée
Empreinte (embedding)	vecteur de quelques nombres résumant une fenêtre de signal ; deux fenêtres semblables donnent des vecteurs proches (distance cosinus)
MCSA	Motor Current Signature Analysis : diagnostic des machines par le spectre de leur courant statorique
Glissement (s)	écart relatif entre la vitesse de synchronisme et la vitesse du rotor d'une machine asynchrone ; les barres cassées signent à $f.(1 \pm 2s)$
Port signal / port stream	deux sémantiques de canaux : valeur maintenue écrasée à chaque publication (signal) ou file de jetons consommés (stream) ; les cycles de flux exigent un retard Z-1 sur les seconds
Nœud fractal	un graphe complet (ici : un modèle ONNX) inséré comme nœud d'un graphe parent ; ses entrées/sorties sont des « ports frontière » nommés
Double banque	technique de mise à jour sûre : le nouveau contenu est validé dans une zone de transit et ne remplace l'ancien que d'un coup, ou pas du tout
Quarantaine d'hystérésis	petite file retenant les derniers échantillons admis par la porte, le temps de savoir s'ils appartiennent déjà au régime suivant
Revue contradictoire	relecture où chaque défaut allégué est confié à un agent chargé de le RÉFUTER par reproduction exécutée ; seuls les défauts qui survivent sont corrigés

Annexe F : inventaire des suites de tests du périmètre

Suite	Objet	Tests
motorwatch.central	catalogue, fusion, protocole, station (y compris rejets atomiques)	15
motorwatch.r385	bout en bout SP1 : 3 régimes, 2 alarmes, diagnostics, fédération	1
motorwatch.induction	bout en bout SP2 : casse à chaud, différentiel, contre-épreuve, fédération	1

Suite	Objet	Tests
motorwatch.pmsm	couverture brushless (agnosticisme moteur de la recette)	1
motorwatch.regime-edge	reproduction du chevauchement de trame + preuve du correctif	1
graphs.roundtrip	chargement headless des 4 graphes, charge échangeable, encodeur poussé dans le graphe	10
ml/online-cluster	open-set, k émergent dont k = 1, _recluster en session réelle, bornes	15
dsp (3 suites)	porte, multiplexeur + fenêtres [T,C], transposition	18
onnx-plugin (2 suites)	sha256 (FIPS), chargements validés, streaming fractal, permutations	30+
physics (loadtorque + induction)	profils signés, passivité, spectre des barres, taux requis, glissement	36
execution/boundary-ports	ports frontière génériques (hors ONNX), fan-out	6

Le mot de la fin

Ce cahier est la contrepartie industrielle de celui de DriverV2 : même exigence de traçabilité (chaque chiffre cité a été mesuré, chaque échec est raconté), même objectif (qu'un ingénieur qui n'a participé à rien puisse tout refabriquer), et une étape de plus dans la méthode : la revue contradictoire exécutable comme condition de sortie. Le système livré découvre, alerte, diagnostique, étiquette et partage ; il attend maintenant son encodeur entraîné et son premier vrai moteur.